

Projet Deep Learning

Reconnaissance et analyse automatique de panneaux de signalisation

Formatrice	Hanane Zerdoum — hanane_zerdoum@yahoo.fr
Équipes	Projet à réaliser en binôme. Les groupes de trois étudiants sont autorisés : un enrichissement conséquent est alors obligatoire.
Outils	Python, Google Colab, Keras / TensorFlow ; YOLO pour les extensions
Livrables	Notebook Colab commenté + présentation + démonstration du modèle final
Soutenance	Vendredi 16 octobre 2026

Contexte

L'objectif de ce projet est de développer progressivement un système de vision par ordinateur capable d'analyser des images de **panneaux de signalisation** et de reconnaître automatiquement leur catégorie.

Selon le jeu de données choisi, les classes pourront par exemple correspondre à des limitations de vitesse, des panneaux de danger, des interdictions, des obligations, des panneaux de priorité ou encore des indications.

Le projet commence par un problème de **classification d'images à l'aide d'un réseau de neurones**, puis il est enrichi au fur et à mesure avec les nouvelles notions de Deep Learning et de vision par ordinateur étudiées pendant le module.

L'objectif final est de passer d'un simple classifieur travaillant sur un panneau déjà extrait à un système capable d'analyser une scène routière, voire une vidéo, contenant un ou plusieurs panneaux.



Figure 1 — Exemples de catégories de panneaux (illustration schématique : les datasets contiennent des photos réelles, prises dans des conditions variées).

Organisation du projet

Ce qui est attendu

TRAVAIL OBLIGATOIRE : parties 1 à 6. C'est le cœur du projet : données, modèle de référence, CNN, entraînement, évaluation, expériences.

ENRICHISSEMENTS / BONUS : parties 7 à 9. Ils sont facultatifs et valorisés ; ils s'appuient sur les notions vues dans les dernières séances.

Binôme : parties 1 à 6 obligatoires ; parties 7 à 9 facultatives et valorisées.

Groupe de 3 : parties 1 à 6 + un enrichissement conséquent obligatoire.

Partie	Contenu	Statut
1	Recherche, compréhension et préparation des données	Obligatoire
2	Modèle de référence (réseau dense)	Obligatoire
3	Construction d'un CNN avec Keras	Obligatoire
4	Entraînement du réseau	Obligatoire
5	Évaluation et analyse des erreurs	Obligatoire
6	Expérimentation et amélioration (≥ 3 expériences)	Obligatoire
7	Enrichissement avec les nouvelles notions du cours	Bonus (facultatif)
8	Détection de panneaux dans une scène (YOLO)	Bonus (facultatif)
9	Extension à la vidéo	Bonus (facultatif)

Avant de commencer — De la classification binaire à la classification multiclasse

Jusqu'à présent, nous avons principalement étudié des problèmes de **classification binaire** : le réseau doit choisir entre deux classes.

Image → réseau de neurones → 1 neurone de sortie → Sigmoid

La fonction Sigmoid fournit une valeur comprise entre 0 et 1, que l'on interprète comme la probabilité d'appartenir à la classe 1.

Dans ce projet, le problème est différent : nous souhaitons reconnaître **plusieurs catégories de panneaux**. Il s'agit donc d'un problème de **classification multiclasse**. Si le dataset contient K classes, le réseau devra produire une sortie pour chacune d'elles :

CNN → K neurones de sortie → Softmax

La fonction Softmax

Softmax transforme les valeurs produites par les neurones de sortie en **probabilités** : elles sont toutes comprises entre 0 et 1 et leur somme est égale à 1.

Par exemple, pour une image donnée et un dataset simplifié à 7 classes, le réseau pourrait produire :

Catégorie	Probabilité
Limitation 30 km/h	0,03
Limitation 50 km/h	0,74
STOP	0,04
Cédez le passage	0,06
Danger	0,05
Sens interdit	0,03
Obligation	0,05
Total	1,00

La probabilité la plus élevée correspond ici à la classe **Limitation 50 km/h** : c'est la classe prédite.

Mathématiquement, si le réseau produit les valeurs $z_1, z_2, \dots$ sur ses K neurones de sortie, la probabilité associée à la classe i est :

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

L'exponentielle rend toutes les valeurs positives, et la division par la somme garantit que les probabilités totalisent 1.

Dans Keras, la dernière couche pourra s'écrire (par exemple $K = 43$ pour le dataset GTSRB) :

```
Dense(K, activation="softmax")
```

Le nombre de neurones de la dernière couche doit donc être adapté au **nombre de classes du dataset choisi**.

À retenir

Classification binaire : 2 classes → 1 neurone de sortie → **Sigmoid**

Classification multiclasse : K classes → 1 neurone par classe (K neurones) → **Softmax**

Partie 1 — Recherche, compréhension et préparation des données

Vous devez rechercher et sélectionner un **jeu de données public de panneaux de signalisation**. Le choix du dataset fait partie du travail demandé.

Description du dataset

Vous présenterez :

- la source du dataset (lien, auteurs, licence ou conditions d'utilisation) ;
- le nombre total d'images ;
- les différentes classes et leur signification ;
- le nombre d'images par classe ;

- les dimensions des images ;
- le format des images (couleur ou niveaux de gris, type de fichier) ;
- la présence éventuelle d'un déséquilibre entre les classes.

Vous afficherez également plusieurs exemples d'images pour différentes catégories de panneaux.

Piste possible

Le benchmark **GTSRB** (German Traffic Sign Recognition Benchmark) est un jeu de classification multiclasse de panneaux routiers comprenant 43 classes et plus de 50 000 images. Vous restez libres de choisir un autre dataset public adapté au projet.

Préparation des données

Avant l'entraînement, vous effectuerez les traitements nécessaires :

- redimensionnement des images (les photos de panneaux n'ont généralement pas toutes la même taille) ;
- normalisation des pixels ;
- préparation des labels ;
- encodage des classes lorsque cela est nécessaire ;
- constitution des ensembles **train / validation / test**.

Vous expliquerez le rôle de chacune de ces étapes. Rappel : le jeu de test est réservé à l'évaluation finale ; il ne doit jamais servir de validation pendant l'entraînement.

Notions nécessaires

Dataset – données d'entrée – labels – classes – train / validation / test – normalisation – encodage des données – dimensions et format des données.

Partie 2 — Construction d'un modèle de référence

Avant de construire un CNN, vous développerez un premier réseau de neurones simple servant de **modèle de référence**. Une possibilité consiste à utiliser :

Image → Flatten → Dense + activation → Dense → Softmax

L'objectif n'est pas d'obtenir d'excellentes performances avec ce modèle : il servira de **point de comparaison** avec le CNN développé ensuite.

Vous devrez être capables d'expliquer :

- comment une image couleur est transformée en entrée du réseau ;
- le rôle des poids et des biais ;
- la propagation avant ;
- les fonctions d'activation ;
- la fonction de perte ;
- le principe de la rétropropagation ;
- la descente de gradient ;
- la sortie du réseau dans le cas d'une classification multiclasse.

Partie 3 — Construction d'un CNN avec Keras

Le schéma ci-dessous rappelle le principe général d'un CNN : une première partie **extraît les caractéristiques** de l'image (convolution, ReLU, pooling), une seconde partie **classe** l'image à partir de ces caractéristiques (Flatten, Dense, Softmax).

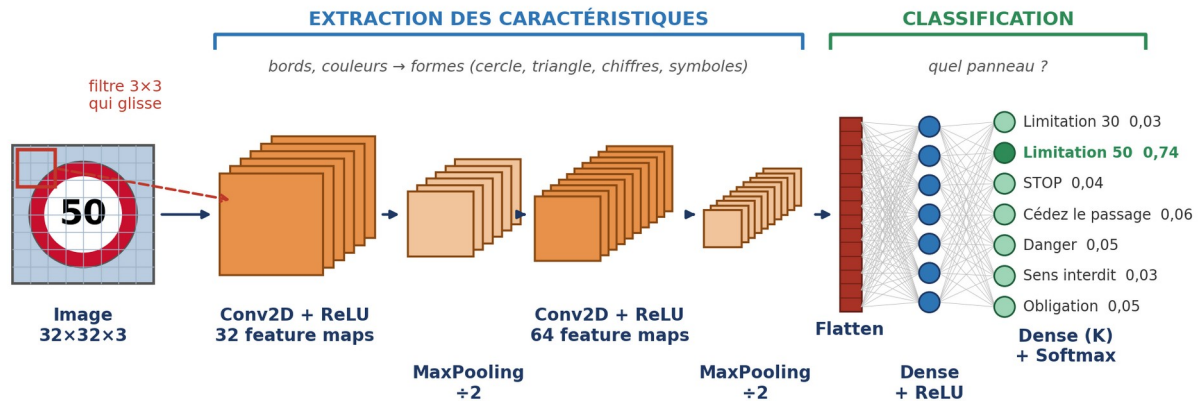


Figure 2 — Principe d'un CNN appliqué à la reconnaissance de panneaux (exemple d'architecture, à adapter).

Vous développerez un **réseau de neurones convolutif (CNN)** adapté au traitement des images de panneaux. Votre architecture devra comporter plusieurs blocs de convolution et de pooling.

Exemple de structure possible :

Image → Conv2D + ReLU → MaxPooling → Conv2D + ReLU → MaxPooling → Flatten → Dense → Softmax

Il ne s'agit pas de reproduire obligatoirement cette architecture : vous devez **proposer et justifier votre propre architecture**. Vous expliquerez notamment :

- le rôle des filtres ;
- le principe de la convolution (sur une image couleur à 3 canaux) ;
- les feature maps ;
- la taille du kernel ;
- le stride ;
- le padding ;
- ReLU ;
- le pooling ;
- Flatten ;
- les couches Dense ;
- la couche de sortie.

Vous suivrez et expliquerez l'évolution des **dimensions des données** tout au long du réseau (par exemple sous forme de tableau couche par couche), ainsi que le nombre de paramètres de chaque couche.

Partie 4 — Entraînement du réseau

Vous entraînerez votre CNN et justifierez notamment :

- la fonction de perte ;

- l'optimiseur ;
- le batch size ;
- le nombre d'époques ;
- les métriques utilisées.

Vous étudierez l'évolution, au fil des époques :

- de la loss d'entraînement et de la loss de validation ;
- de l'accuracy d'entraînement et de l'accuracy de validation.

Vous interpréterez ces courbes et identifierez un éventuel **surapprentissage**.

Partie 5 — Évaluation et analyse des erreurs

L'accuracy seule n'est pas suffisante, en particulier lorsque les classes sont déséquilibrées. Vous produirez une **matrice de confusion** et étudierez les performances obtenues pour les différentes catégories de panneaux.

Vous chercherez à déterminer :

- quels panneaux sont correctement reconnus ;
- quels panneaux sont fréquemment confondus (par exemple entre différentes limitations de vitesse) ;
- quelles classes semblent les plus difficiles à apprendre, et pourquoi ;
- si certaines erreurs peuvent être liées à la luminosité, à l'angle de vue, à la taille du panneau, au flou ou à la ressemblance entre classes.

Vous afficherez plusieurs exemples sous la forme :

image + vraie classe + classe prédite + probabilité

Vous analyserez également plusieurs prédictions incorrectes.

Partie 6 — Expérimentation et amélioration

Vous réaliserez **au minimum trois expériences** afin d'améliorer ou d'étudier votre modèle. Pour chaque expérience, modifiez un nombre limité de paramètres à la fois afin de pouvoir interpréter l'effet observé.

Vous pourrez notamment modifier : le nombre de couches convolutionnelles, le nombre de filtres, la taille des kernels, le pooling, le nombre de neurones, le batch size, le learning rate, l'optimiseur ou le nombre d'époques.

Vous pourrez également utiliser des techniques telles que : Dropout, Data Augmentation, Early Stopping, régularisation.

Point d'attention — Data Augmentation

Toutes les transformations ne sont pas adaptées aux panneaux. Un **retournement horizontal** transforme par exemple « obligation de tourner à gauche » en « obligation de tourner à droite » : l'image change de classe, mais pas son label. Choisissez et justifiez des transformations qui conservent le sens du panneau (légère rotation, variation de luminosité, zoom...).

Vous présenterez vos expériences dans un **tableau comparatif** (modification apportée, résultats sur la validation, observations) et justifierez le choix du modèle final.

Partie 7 — Enrichissement avec les nouvelles notions du cours

Le projet est **évolutif**. Au fur et à mesure des notions étudiées pendant le module, vous réfléchirez à la manière dont elles peuvent enrichir votre système. Vous pourrez notamment explorer :

- Data Augmentation ;
- Transfer Learning et réseaux pré-entraînés ;
- régularisation ;
- détection d'objets ;
- YOLO.

Partie 8 — Extension : détection de panneaux dans une scène

Dans les premières parties, le système travaille sur une image contenant déjà principalement un panneau :

Panneau déjà extrait → CNN → classe prédite

L'objectif est maintenant de traiter une scène routière complète, dans laquelle un ou plusieurs panneaux peuvent apparaître. La chaîne de traitement devient :

Image → Détection des panneaux → Bounding boxes → Extraction → CNN → Classes prédites

Cette étape fait passer de la **classification** (une réponse pour toute l'image) à la **détection** (localiser chaque panneau dans l'image) :

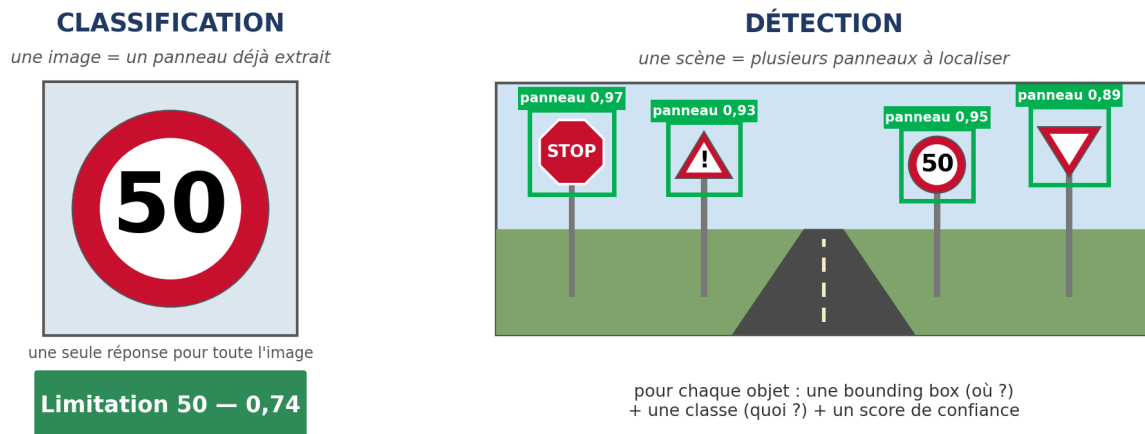


Figure 3 — Classification vs détection : à gauche, une seule réponse pour l'image ; à droite, une bounding box et un score pour chaque panneau.

Un modèle tel que **YOLO** pourra être utilisé pour la partie détection. Le résultat final affichera, pour chaque panneau : une bounding box, la classe prédite et le score associé.

Points d'attention

Choix du détecteur. Un modèle YOLO pré-entraîné standard (dataset COCO) ne connaît qu'un seul type de panneau, le STOP. Il vous faudra donc utiliser un modèle entraîné à détecter des panneaux, ou l'adapter (fine-tuning) sur un dataset de détection de panneaux, et justifier votre choix.

Cohérence du prétraitement. Chaque panneau extrait doit subir exactement le même prétraitement que les images d'entraînement de votre CNN (taille, couleur, normalisation). Sinon, les prédictions n'ont pas de sens.

Deux modèles, deux rôles. Expliquez clairement ce que fait le détecteur (où est le panneau ?) et ce que fait votre CNN (quel est ce panneau ?).

Partie 9 — Extension possible à la vidéo

Pour aller plus loin, le système pourra être testé sur une courte vidéo ou une séquence d'images représentant une route :

Vidéo → images successives → détection → extraction des panneaux → CNN → prédictions → affichage

L'objectif n'est pas de développer un système de conduite autonome, mais de montrer que la chaîne de traitement fonctionne sur plusieurs images successives.

Notions de YOLO utiles pour l'extension

Pour cette extension, vous aurez notamment besoin de comprendre :

- la différence entre classification et détection ;
- bounding box ;
- confidence score ;
- Intersection over Union (IoU) ;
- Non-Maximum Suppression (NMS) ;
- le principe général de YOLO ;
- la notion de modèle pré-entraîné ;
- éventuellement Transfer Learning / Fine-tuning ;
- precision, recall et mAP.

Il n'est pas demandé de reconstruire YOLO à partir de zéro. L'objectif est de comprendre son fonctionnement général et de savoir **l'intégrer à votre projet**.

Pipeline final possible

Le schéma ci-dessous résume le système complet visé par le projet. La partie droite (CNN + Softmax) correspond aux parties 1 à 6 ; la partie gauche (détection avec YOLO) correspond aux extensions des parties 8 et 9.

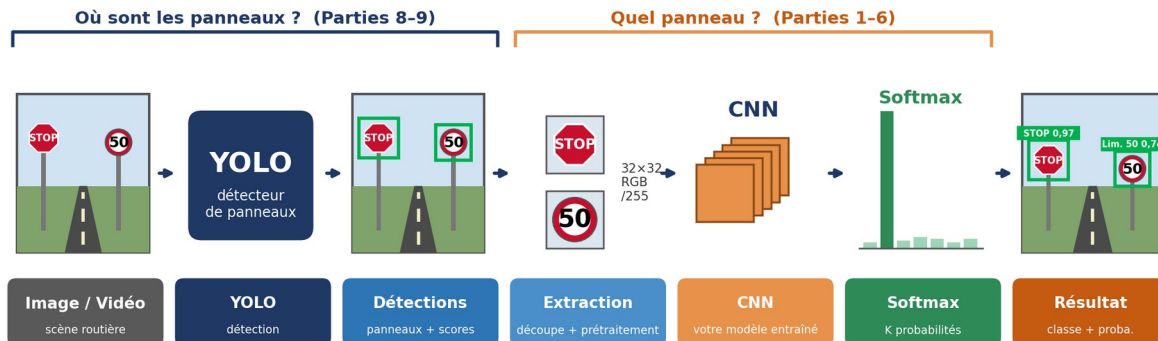


Figure 4 — Pipeline complet du projet : Image/Vidéo → YOLO → panneaux détectés → extraction → CNN → Softmax → classe + probabilité.

Livrables

Lors de la soutenance du **vendredi 16 octobre 2026**, chaque équipe devra fournir :

1. **Un notebook Python / Google Colab** structuré, fonctionnel et commenté, qui s'exécute de bout en bout.
2. **Une présentation** de la démarche, des architectures, des expériences et des résultats.
3. **Une démonstration** du modèle final.

Conseils pour le notebook

- organiser le notebook en sections correspondant aux parties du projet ;
- commenter les choix importants plutôt que chaque ligne de code ;
- afficher clairement les dimensions des données et les résultats ;
- supprimer les cellules inutiles ou non exécutées ;
- vérifier que le notebook peut être relancé dans l'ordre, du début à la fin.

Barème

Le projet est noté sur 20 points. Les **18 premiers points** correspondent au socle obligatoire (parties 1 à 6 et soutenance) ; les **2 derniers points** valorisent l'enrichissement (parties 7 à 9). La répartition est la suivante :

Évaluation	Parties concernées	Points
Données et préparation	Partie 1	3
Modèle de référence + CNN	Parties 2 et 3	5
Entraînement et évaluation	Parties 4 et 5	3
Expérimentations et analyse critique	Partie 6	4
Enrichissement : valorisation des améliorations et extensions mises en œuvre	Parties 7 à 9	2
Soutenance, démonstration et maîtrise du travail	Soutenance	3
Total		20

Pour les groupes de trois étudiants, un enrichissement conséquent est obligatoire (par exemple la détection de panneaux dans une scène avec YOLO, partie 8). Il est évalué dans la ligne « Enrichissement » du barème.

Soutenance — vendredi 16 octobre 2026

Chaque membre de l'équipe devra être capable d'expliquer :

- le fonctionnement du système ;
- l'architecture du CNN ;
- les principales parties du code ;
- les choix réalisés ;
- les résultats obtenus ;
- les erreurs observées ;
- les améliorations apportées.

Des questions pourront être posées directement sur le notebook ou sur certaines parties du code, à n'importe quel membre de l'équipe.

Objectif final

Ce projet permet de relier progressivement toutes les notions du module :

Neurone → MLP → Backpropagation → Keras → CNN → Convolution → Classification multiclasse → Softmax → Amélioration du modèle → Détection → YOLO → Application complète de vision par ordinateur